

Track 1 — Malicious Contract Dark Patterns: Literature & Taxonomy

19 September 2026

Status: research note (input to the detector spec; not itself the spec)
Date: 2026-09-19
Scope: TRUST404 Track 1, *Smart Contract Threat Detection* — offline `.sol` directory in, Benign / Malicious / Uncertain + evidence JSON out
Anchor: trust404.co.kr/tracks

0. What the track is grading

From the track page, verbatim constraints that shape everything below:

- Input: `.sol` source, **directory batch mandatory**, single file optional.
- Output: clear verdict (Benign / Malicious / Uncertain) + 근거 (function/code location + reasoning) + machine-readable JSON.
- **Offline grading.** No network. External API / LLM calls do not run. Ship models and rules inside the submission.
- Non-interactive entry point (CLI / script / `docker run`).
- Judged on: mock contracts (public + **private** set); identifying risk from **logic flow and permission structure, not keyword matching**; sound reasoning — “비정상적 권한 이전, 은닉된 자산 인출 경로”.

Those two Korean phrases are literal category names in the rug-pull taxonomies below (*Ownership Fraud*; *Leaking Token / Funds Manipulation*). The problem class is **scam-contract detection**, not vulnerability detection.

1. Problem framing the literature agrees on

Three sub-literatures, all about **deployer intent**, not accidental bugs:

Sub-field	Core question	Era
Honeypots	Contract baits a user/attacker into depositing, then traps funds	2019–
Backdoors / rug pulls in tokens	Privileged functions let the deployer freeze, drain, mint, or tax users	2022–
Trapdoors	“You can buy but you cannot sell”	2023–

Every strong tool reduces to one structural question:

*Is there a **privileged writer** to a **state variable** that sits on the user’s **transfer / exit path**, or that touches **balances** directly?*

That is the privilege + exit-path design. The papers validate it and supply the exact predicates (§5).

2. Paper map

2.1 Foundational

HoneyBadger — “The Art of the Scam” · Torres, Steichen, State · USENIX Security 2019

[PDF](#) · [arXiv](#)

First honeypot taxonomy: 8 techniques in 3 levels (EVM / Solidity compiler / Etherscan). Symbolic execution on bytecode; 690 honeypots, 87% precision on manual validation. **Appendix Table 5 lists 24 named honeypots with addresses** — usable fixtures.

Pied-Piper · Ma et al. · TOSEM 2022

[ACM](#) · [repo](#) · [tool](#)

Five ERC-token backdoors: *Arbitrary Transfer*, *Generate Token After ICO*, *Destroy Token*, *Disable Transferring*, *Freeze Account*. Datalog + directed fuzzing. 13,484 contracts → 189 confirmed, 4 CVEs (incl. CVE-2019-16944). Built **200 backdoor-injected contracts** as a test set — ideal for fixtures.

TokenScope · Chen et al. · CCS 2019

[PDF](#)

Detects inconsistency between storage changes, interface behavior, and events (mint without `Transfer`, `balanceOf` that lies). >99.9% precision. Basis for the “view function that branches on caller” check.

2.2 Rug-pull static detection (closest to this build)

CRPWarner · Lin et al. · IEEE TSE 2024

[arXiv](#) · [repo](#) + [datasets](#)

103 real rug pulls → contract-related types: **Hidden Mint Function**, **Limiting Sell Order**, **Leaking Token** (incl. unlimited fee). Datalog rules on bytecode. P 91.8 / R 85.9 / F1 88.7 on 69 open-source scam contracts; 84.9% precision at scale (4,168 flagged of 13,484). Has an explicit false-positive analysis (§6).

Rules (Formula 1–4):

- **Hidden Mint**: $\text{PublicFuncForOwner}(f) \wedge \text{LoadAndStoreBalances}(f) \wedge \neg \text{CheckBalances}(f)$
- **Limiting Sell**: $\text{PublicFuncForOwner}(f) \wedge \text{VarToLimitTransfer}(v) \wedge \text{FuncModifyStorage}(f, v)$
- **Leaking Token**: $\text{PublicFuncForOwner}(f) \wedge \text{FuncTransfer}(f) \wedge \text{CheckBalancesOfInput}(f)$
- **Unlimited fee**: $\text{PublicFuncForOwner}(f) \wedge \text{VarForFee}(v) \wedge \text{FuncModifyStorage}(f, v)$ with no bound

Tokeneer — “Stop Pulling my Rug” · Zhou, Sun, Ma, Chen, Yan, Jiang (Tsinghua) · ICSE-SEIP 2024

[PDF](#) · [ACM](#) · [repo](#) github.com/TokenSecure/Tokeneer

201 incidents (\$425M) → four risks with base rates: **BlackList 58.7%**, **ModifyBalance 42.3%**, **TimeLimit**, **AlienDepend** (external contract controls transfer). Models the transfer as 4 sub-processes (`CallPublicTransfer` → `EnterInternalTransfer` → `SenderBalanceReduction` → `ReceiverBalanceAddition`) + 5 plugins (`AddrCheck`, `TimingCheck`, `InternalCall`, `ExternalCall`, `BreakIn`) and matches oracles **anywhere on the reachable path**. 98.0% recall / 98.9% precision; found 27.2% more real risks than GoPlus and Pied-Piper. Key insight: Pied-Piper and GoPlus miss checks placed *before* `_transfer` is entered (Dialectic) or wrapped in helpers (`getCheckRoot` in TMK).

TrapdoorAnalyser — “From Programming Bugs to Multimillion-Dollar Scams” · Huynh et al. · 2023, rev. Dec 2024

[arXiv v4](#)

Manually inspected 1,859 trapdoor tokens. Five techniques: **Exchange Permission, Exchange Suspension, Amount Limit, Fee Manipulation, Invalid Callback**. Semantic check is **built on Slither’s AST via its Python API** — same stack as this project. Dataset 30k labeled UniswapV2 tokens (11,943 trapdoor / 18,548 non). GoPlus detected only **60%** of their trapdoors.

Their semantic check collects: state vars s (address / int / bool / const), transfer functions F + everything they call, transfer inputs $I = \{sender, receiver, amount\}$, end nodes N (require / assert / revert / return) used in F , backdoor functions B (caller compared with a stored address or list), external calls c . An indicator = a var in s used in an end node of F (or in the `amount` computation) **and** writable from B or c .

RPHunter — “Your Token Becomes Worthless” · Wu et al. · 2025

[arXiv](#) · [dataset \(IEEE DataPort\)](#) · [Figshare](#)

Largest manual corpus: **645 rug pulls with source**. Code-risk taxonomy, 3 categories / 8 sub-types, with counts:

- Sale Restrict (203): Amount Restrict, Timestamp Restrict, Address Restrict
- Variable Manipulation (144): Modifiable Tax Rate, Modifiable Tax Address, Modifiable External Call
- Balance Tamper (189): Hidden Mint/Burn, Hidden Balance Modification

Detecting Rug-Pull via balance tracking · Applied Sciences 2025

[MDPI](#)

Tracks balance changes per function on bytecode; six backdoor types: token generation, destruction, transfer limitation, funds manipulation, transfer fee, proxy. 0.98 precision / 0.96 recall.

Geth-based ERC20 honeypot detector · Discover Computing 2025

[Springer](#)

Taint: boolean storage indexed by `sender/to` that decides a revert branch → honeypot. Missed the 65/189 Pied-Piper backdoors that don’t touch `transfer` (mint / destroy / arbitrary transfer) — a reminder that exit gating alone is not enough.

2.3 Taxonomies and tool audits

SoK: A Taxonomic Analysis of DeFi Rug Pulls · Sun, Ma, Nie, Liu (NTU) · ISSTA 2025

[PDF](#) · [ISSTA page](#)

31 papers + 27 industry sources → **35 rug-pull types**, 9 new (Hidden Mint, Burn, Balance Modification, Ownership Transfer, Hidden Owner, Unverified Contract, External Call, Liquidity Pool Block, Fake LP Lock). 13 tools evaluated: coverage **25.7%–62.9%**; 9 types undetected by any tool; on compound scams best tool drops to **31.3%**. Case study DOGE3.0: proxy hiding ownership + `_sender` hidden governance + “burn” that mints. Names four static-tool failure modes (§4).

SoK: Rug Pull Causes, Datasets, Tools · 2024 (earlier version)

[arXiv](#)

34 root causes. Types no tool covered: Fake LP Lock, Hidden Fee, Destroy Token, Fake Money Transfer, Ownership Transfer, Liquidity Pool Block, Freeze Account, Wash-Trading, Hedge.

ObfProbe — “Obfuscated Funds Transfers” · 2025

[arXiv](#)

Seven bytecode-level obfuscation features on transfer logic across 1.03M contracts; 3,128 heavily obfuscated. A SOTA Ponzi detector fell **79% → 12%** under obfuscation. Documents fake `renounceOwnership()`, `failsafe()` / `emergency()` backdoors, `isTokenReceiver` hidden owner.

Static analysis of NFT rug pulls · 2025 · [arXiv](#)

Slither at scale over 49.9k NFT contracts; most flagged constructs: owner-only withdraw, unrestricted mint, selfdestruct, delegatecall, tx.origin auth, deceptive names (safeWithdraw).

2.4 Transaction-side (context only — not usable in offline grading)

- **Trade or Trick?** · Xia et al. · SIGMETRICS 2022 · [arXiv](#) — 10,920 scam tokens ≈ **50% of Uniswap V2**; guilt-by-association + ML on tx data.
- **Token Spammers, Rug Pulls, and Sniper Bots** · Cernera et al. · USENIX Security 2023 · [page](#) · [dataset](#) — 60% of tokens live < 1 day; “1-day rug pull” ≈ \$240M.
- **Do Not Rug On Me** · Mazorra, Adan, Daza · 2022 · [arXiv](#) — ML on pool / holder / tx-graph features, 27,588 tokens.
- **Detecting Ponzi Schemes on Ethereum** · Chen et al. · WWW 2018 · [PDF](#) — opcode-frequency + account features, XGBoost.

2.5 Industry checklists (what most competitors will copy)

- **GoPlus token security fields** · [docs](#): `is_honeypot`, `hidden_owner`, `can_take_back_ownership`, `is_mintable`, `slippage_modifiable`, `personal_slippage_modifiable`, `transfer_pausable`, `is_blacklisted`, `is_whitelisted`, `is_anti_whale`, `trading_cooldown`, `is_proxy`, `external_call`, `selfdestruct`, `owner_change_balance`.
- **TokenSniffer exploit typologies** · [docs](#): `honeypot`, `blocklist/allowlist`, `hidden mint`, `balance modification`, `fake ownership renounce`, `fee modifier`. Tests: `testForProxy`, `testForPausable`, `testForMint`, `testForRestoreOwnership`, `testForMaxTransactionAmount`, `testForModifiableFee`, `testForBlacklist`, `testForOwnershipNotRenounced`.
- **ethereum.org token integration checklist** · [source](#) — “Owner privileges”: not upgradeable, limited mint, not pausable, cannot blacklist.
- **Ice phishing / drainers** · [Forta](#) · [Check Point](#) — `approve` / `permit` / `setApprovalForAll` harvested via fake claim pages, then `transferFrom` within the same block, often via `multicall`.

3. Consolidated dark-pattern taxonomy

Merged from HoneyBadger, Pied-Piper, CRPWarner, Tokeneer, TrapdoorAnalyser, RPHunter, ISSTA SoK, GoPlus, TokenSniffer. Grouped by **what the attacker gains**. Real-world example tokens in parentheses are from the cited papers.

A. Exit gating — Sale Restrict / Limiting Sell Order / Trapdoor

Most common family: 58.7% of Tokeneer incidents; 203/645 in RPHunter.

Pattern	Shape	Example
Address restrict	<code>mapping(address=>bool)</code> read in a <code>require/if/revert</code> on the transfer path; privileged-writable. Names lie: <code>_isBot</code> , <code>isExcluded</code> , <code>_antiswaplist</code> , <code>_safeOwner</code>	AquaDrops, Sirius_Finance
Global suspension	<code>bool</code> (<code>tradingEnabled</code> , <code>transfersEnabled</code> , <code>paused</code> , or a one-letter name) gating transfer; privileged-writable. Worse if no public un-gate exists	Connective, Bancor (legit use)
Amount limit	<code>maxTxAmount</code> / <code>maxWallet</code> compared with amount in an end node; privileged-writable with no floor	EVGR (set to 1)
Timestamp restrict / cooldown	<code>block.timestamp</code> compared with a privileged-writable limit	Tokeneer TimeLimit
Sell-only branch	Gate applies only when <code>to == uniswapV2Pair</code> (or any privileged-writable address var) — buy works, sell reverts	most trapdoors

Pattern	Shape	Example
Invalid callback	Transfer triggers a nested call that re-enters the gate with a controlled address so sells always revert while the seller is never listed	ELONAJA (burnToken → <code>_transfer(token, ...)</code>)
Fee weaponization	Fee var used to compute amount; privileged-writable; no cap. Extreme values cause underflow that looks like a bug	88 Dollar Millionaire (99%), CPP4U (1000%)

B. Balance tamper — Hidden Mint / Balance Modification / ModifyBalance

42.3% of Tokeer incidents; 189/645 in RPHunter.

Pattern	Shape	Example
Privileged mint, arbitrary target	<code>balances[x] += n / totalSupply</code> += n in a privileged function, no sufficiency check, no cap, outside constructor	Pokemoney / NEKOGOLD, TEDDY
Mint disguised as burn	“burn” that, on a condition, increases the owner’s balance	DOGE3.0
Mint inside transfer	Credit on the transfer path exceeds the debit, conditioned on a “special” recipient	ISSTA SoK Listing 2
Caller-dependent view	<code>balanceOf / totalSupply</code> branch on <code>msg.sender</code> (dual state)	ISSTA SoK Listing 3; TokenScope class
Privileged burn from other account	<code>balances[from] -= n</code> in privileged fn, from arbitrary	Pied-Piper type 3, CVE-2019-16944
Direct balance set	<code>balances[x] = v</code> in privileged fn	TokenSniffer “balance modification”

C. Fund extraction — Leaking Token / Arbitrary Transfer / Funds Manipulation

The track’s “은닉된 자산 인출 경로”.

Pattern	Shape	Example
Arbitrary transferFrom	Privileged fn moves tokens from any account without allowance	Pied-Piper zero_fee_transaction; CRPWarner Leaking Token
Exempt privileged path	Branch in <code>_transfer</code> for a privileged sender that skips balance check / debit	ZHONGHUA
Privileged sweep	Sends contract-held ETH/tokens out; named emergencyWithdraw, failsafe, rescue, safeWithdraw	Gold Mine Finance
Mutable fee recipient	Fee/tax address is privileged-writable	RPHunter MTA
Honeypot bait (legacy)	Deposit accepted; exit path has hidden owner-only condition — Hidden Transfer, Straw Man Contract, Balance Disorder, Inheritance Disorder, Skip Empty String Literal, Type Deduction Overflow, Uninitialised Struct, Hidden State Update	HoneyBadger Table 5

D. Control-plane deception — Ownership Fraud

The track’s “비정상적 권한 이전”.

Pattern	Shape	Example
Hidden owner	A second address state var (<code>_sender</code> , <code>isTokenReceiver</code> , <code>_safeOwner</code> , <code>dev</code> , <code>marketingWallet</code>) compared with <code>msg.sender</code> in an auth check while owner is visibly renounced	DOGE3.0; only 1/5 tools caught <code>_sender</code> (ISSTA SoK)
Fake renounce	<code>renounceOwnership()</code> emits the event but does nothing, or moves ownership to another attacker address, or leaves the hidden role intact	ObfProbe cases
Take-back ownership	Non-standard fn assigns owner; post-deploy initialize	GoPlus can_take_back_ownership
tx.origin auth	Authorization via <code>tx.origin</code>	NFT rug-pull study
Incomplete renouncement	Second privileged address added before renouncing	DYDZ

E. Structural escape hatches — AlienDepend / Modifiable External Call / Proxy

Pattern	Shape	Note
External gate	Transfer gate or logic delegated to an external contract whose address is privileged-writable (<code>_antiBot.getAntiBotEnable()</code> , hex-named fns in <code>botProtection</code>)	Cannot resolve offline → Uncertain with reason , never Benign (Elongate Deluxe, YZZ)
Settable <code>delegatecall</code>	Target address writable by privileged role	
Upgradeable proxy, EOA admin	Single-key upgrade authority	DOGE3.0 used a proxy to hide ownership
<code>selfdestruct</code>	Present and reachable	

F. Approval-phishing / drainer contracts

Relevant because the track’s user is in a **sign popup**.

Pattern	Shape
Claim/airdrop drainer	Contract whose real effect is <code>transferFrom/permitTransferFrom/safeTransferFrom</code> from <code>msg.sender</code> to a hardcoded or owner-set address, often via <code>multicall</code> , with no matching credit to the caller. Selectors: <code>0x23b872dd</code> , <code>0xd505accf</code> , <code>0xa22cb465</code>

G. Ponzi / scheme contracts

Payouts to earlier depositors funded only by later `msg.value`; no external value source. Cheap structural check if the mock set includes it; low priority.

4. Where traps hide — why keyword / selector matching fails

From TrapdoorAnalyser §3.6, Tokeer §3, ObfProbe, ISSTA SoK.

Placement

- In a modifier attached mid-signature and defined in another contract (YearnLending.Finance `onlyPermitted`).
- In a helper 2–3 layers deep (`getCheckRoot`); fee computed across nested functions (The Reckoning Force).
- In a check placed *before* `_transfer` is entered (Dialectic).
- In an external contract (Elongate Deluxe).

Naming and presentation

- Blacklist named `_isBot`; pair address named “zero address”; one-letter switches (`i`, `t`, `l`).
- Innocent names on backdoors: `failsafe`, `emergency`, `isTokenReceiver`, `safeWithdraw`.
- Dummy `init` that would set the var correctly but is never called (AIRSHIB).
- Blank revert messages.
- Numerical exception instead of `require` so failure reads as an honest bug.

Code structure (ObfProbe’s seven features)

- Dead code, deep function splitting, inline assembly / raw `sstore`, string ops to compute addresses, low transfer-instruction ratio in the transfer function.

Runtime-conditional activation

- Gated on `block.timestamp`, a special recipient, or a proxy’s current implementation.

The ISSTA SoK's four failure modes of static tools

1. Dynamic-context blindness (timestamp-gated behavior)
2. Hidden-variable resolution (dynamically assigned owner addresses)
3. Complex conditional logic (multi-condition gates)
4. Governance-functionality decoupling (the check and the balance op live in different functions)

The convergent fix (Tokeer, TrapdoorAnalyser, RPHunter): model the transfer path as the call graph reachable from `transfer / transferFrom` (including modifiers and pre-checks), then ask what **data** gates it and who **writes** that data. Never match names.

5. The unifying formula → implementation predicates

Every strong tool reduces to the same triple:

1. **Privileged writer ϵ** — a public/external function whose reachability depends on `msg.sender == <state address var>, <address→bool map>[msg.sender]`, or `tx.origin`, in any modifier or inline check. **Name-agnostic** (catches hidden owners).
2. **Gated or tampered state ν** — a state variable written in ϵ .
3. **Impact** — ν is
 - (a) read in a `require / if / revert` end node on the transfer path → exit gating
 - (b) used in the computation of the transferred or fee amount → fee manipulation
 - (c) itself the `balance / totalSupply` mapping written directly → balance tamper
 - (d) an address used as the target of an external call / `delegatecall` on the transfer path → alien dependence

Slither (Python API) supplies every predicate:

Predicate	Slither surface
Privileged writer	<code>Function.is_protected()</code> , modifier bodies, <code>msg.sender / tx.origin</code> comparisons in <code>require/if</code> nodes, data dependency on <code>msg.sender</code>
State written	<code>Function.state_variables_written</code> , <code>all_state_variables_written()</code>
Transfer path	call graph reachable from <code>transfer / transferFrom</code> + their modifiers (<code>Function.all_internal_calls()</code> , <code>Function.modifiers</code>)
End nodes	nodes containing <code>require / assert / revert (SolidityCall)</code> , <code>if</code> conditions
Amount dependency	<code>is_dependent(var, amount_param, function)</code>
Balance mapping	the mapping read by <code>balanceOf / debited</code> in transfer (bind it; do not treat any <code>address→uint</code> as balances)
External / delegatecall	<code>SlithIR HighLevelCall</code> , <code>LowLevelCall</code> , <code>delegatecall</code>
Views that lie	view functions whose return depends on <code>msg.sender</code>

TrapdoorAnalyser built exactly this on Slither. This project re-implements a published design with a clearer verdict layer and offline packaging.

6. Benign lookalikes — what stays Uncertain, not Malicious

Where GoPlus-copying competitors lose points on 정상 samples.

Lookalike	Why it is confused	Discriminator
<code>_burn</code> vs <code>mint</code>	CRPWarner FP: no balance check + compiler rewrote SUB as ADD	Source-level operator is visible — use it; a mint <i>increases</i> the target

Lookalike	Why it is confused	Discriminator
excludeFromFee vs blacklist	Same shape: address→bool, owner-writable, read in <code>_transfer</code>	What the read feeds : fee amount (benign) vs an end node that reverts (gate)
DEX pair check vs onlyOwner	<code>to == uniswapV2Pair</code> looks like an address compare	Only <code>msg.sender / tx.origin</code> comparisons are auth
address→uint status maps	Mistaken for balances	Bind “balance mapping” to the one <code>balanceOf</code> reads
OpenZeppelin Pausable, Ownable, ERC20Capped, AccessControl	Legit and ubiquitous. Lido LDO has <code>transfersEnabled + owner-only enableTransfers</code> ; Bancor shipped <code>disableTransfers</code>	Single bounded privilege with a public un-gate → Uncertain / Benign-with-note. Unbounded, hidden-owner, or combined with balance tamper → Malicious
Anti-whale / anti-bot launch limits	Same vars as Amount Limit	Hard-coded floor + time-limited window → benign-shaped; no floor, no expiry → trapdoor
Rescue of <i>foreign</i> tokens	Looks like a sweep	Sweep of tokens the contract itself issues / holds for users is the malicious case

Calibration from base rates: exit gating and balance tamper explain most incidents in every corpus. Fee-only and pause-only findings are the ones most often present in benign samples.

7. Draft rule set (for the detector spec)

Severity: **HIGH** → drives Malicious; **MED** → drives Uncertain; **INFO** → evidence only.

Rule ID	Family	Trigger	Severity
PRIV_ROLE	meta	Name-agnostic privileged role: <code>msg.sender / tx.origin</code> compared with stored address or <code>address→bool</code> map in an auth check	— (feeds others)
EXIT_ADDR_GATE	A	Privileged-writable <code>address→bool</code> read in an end node on the transfer path	HIGH
EXIT_GLOBAL_SWITCH	A	Privileged-writable <code>bool</code> gating the transfer path	HIGH if no public un-gate; MED if OZ-Pausable shape with un-pause
EXIT_AMOUNT_LIMIT	A	Privileged-writable numeric compared with amount in an end node	HIGH if no floor; MED if constant floor
EXIT_TIME_GATE	A	<code>block.timestamp</code> compared with privileged-writable limit on transfer path	MED; HIGH if no expiry
EXIT_SELL_ONLY	A (modifier)	Gate applies only when <code>to</code> equals a privileged-writable address var	escalate one level
EXIT_CALLBACK_CYCLE	A	Call cycle on transfer path whose input is compared with a privileged-writable var in an end node	HIGH
FEE_UNBOUNDED	A/C	Privileged-writable fee var feeding <code>amount</code> , no cap $\leq 25\%$	HIGH; MED if capped
FEE_ADDR_MUTABLE	C	Privileged-writable fee recipient	MED
BAL_PRIV_MINT	B	Privileged write increasing balance/totalSupply, arbitrary target, no cap, outside constructor	HIGH; MED if ERC20Capped shape
BAL_PRIV_BURN_OTHER	B	Privileged decrease of another account's balance	HIGH
BAL_DIRECT_SET	B	Privileged direct assignment to balance mapping	HIGH
BAL_TRANSFER_HIDDEN_MINT	B	Credit > debit on transfer path	HIGH

Rule ID	Family	Trigger	Severity
VIEW_CALLER_DEPENDENT	B	balanceOf / totalSupply return depends on msg.sender	HIGH
LEAK_ARBITRARY_TRANSFERFROM	C	Privileged move from arbitrary account without allowance	HIGH
LEAK_EXEMPT_PATH	C	Privileged branch in transfer that skips debit/check	HIGH
LEAK_PRIV_SWEEP	C	Privileged send of contract-held ETH / self-token	HIGH if contract custodies user funds; MED if foreign-token rescue only
OWN_HIDDEN_ROLE	D	Auth address var that is not the visible owner	HIGH
OWN_FAKE_RENOUNCE	D	renounceOwnership does not clear all auth vars, or reassigns	HIGH
OWN_REASSIGN_NONSTD	D	owner assigned outside constructor / standard transferOwnership	HIGH
OWN_TX_ORIGIN	D	tx.origin used for auth	MED
STRUCT_EXTERNAL_GATE	E	Transfer path depends on external call to privileged-writable address	→ Uncertain with reason
STRUCT_DELEGATECALL_SETTABLE	E	delegatecall to privileged-writable target	HIGH
STRUCT_SELFDESTRUCT	E	selfdestruct reachable	HIGH
STRUCT_PROXY_EOA_ADMIN	E	Upgradeable with single-address admin	MED
DRAIN_APPROVAL_PULL	F	transferFrom / permit / safeTransferFrom from msg.sender to non-caller with no credit back	HIGH
HONEYPOT_LEGACY_*	C/F	HoneyBadger patterns (hidden owner condition on exit, straw man, etc.)	MED
PONZI_SHAPE	G	Payout to stored depositors funded only by msg.value	MED
SLITHER_HIGH_OVERLAY	—	Slither built-in High detectors	INFO; can lift Benign → Uncertain only

Draft verdict policy (tune on Discord public samples):

- any HIGH → **Malicious**
- only MED → **Uncertain**
- STRUCT_EXTERNAL_GATE or compile failure → **Uncertain** with reason
- nothing → **Benign**
- Optional: two MED from different families (A + B, A + D) → Malicious

Every finding carries contract, function, source_lines, rule_id, severity, reasoning.

8. Datasets for fixtures and tuning

Source	What	Link
Pied-Piper	200 backdoor-injected contracts + real cases	repo, tool
CRPWarner	69 open-source rug pulls (ground truth) + 13k large set	repo
Tokeer	code + data	github.com/TokenSecure/Tokeer

Source	What	Link
TrapdoorAnalyser	30k labeled UniswapV2 tokens	link in paper §4.3
RPHunter	645 rug-pull sources	IEEE DataPort , Figshare
ISSTA 2025 SoK	2,391 instances across 29/35 types	paper artifact
HoneyBadger	24 named honeypots with addresses (Table 5)	PDF
H6-Guard	Solidity → JSON-verdict honeypot dataset	Hugging Face
Cernera et al.	Ethereum/BSC token + LP dataset	repo

Priority: **Discord public samples first** — that is the judges’ mock style. Pull external fixtures while network is still available; grading is offline.

9. Implications for the build

Direction unchanged; three additions justified by the papers:

1. **Name-agnostic privilege detection** — any address state var or `address→bool` map compared to `msg.sender / tx.origin` in a reachable auth check. Biggest gap in every tool the SoK tested.
2. **Transfer-path modeling à la Tokeer** — collect every function/modifier reachable from `transfer / transferFrom`, including pre-`_transfer` checks; evaluate gates on that set, not on `_transfer` alone.
3. **Explicit Uncertain-with-reason** for unresolved external calls on the transfer path and for compile failures, so “we cannot see the logic” is a scored answer, not a silent Benign.

Source-only input is an advantage here: CRPWarner, Tokeer, and RPHunter had to decompile bytecode; this project gets Slither’s AST/IR directly, like TrapdoorAnalyser.

10. Decisions log

- **Offline-only submission path.** No Tenderly, no mainnet `eth_call`, no fork, no hosted model (incl. TypeSafe Jev). Those may appear in the Demo Day story only.
- **Slither as a library, not as the product.** Verdict logic and evidence are ours; Slither is the parser/IR.
- **If simulation is used, it is local Anvil/Foundry on the compiled mock contract** (privileged `setX` then user `transfer`), inside `docker run --network none`.
- **Slither built-in detectors are an overlay**, never the driver of Malicious.